

Séance 10

Aujourd'hui

- L'héritage
- Les méthodes virtuelles

L'héritage : définition

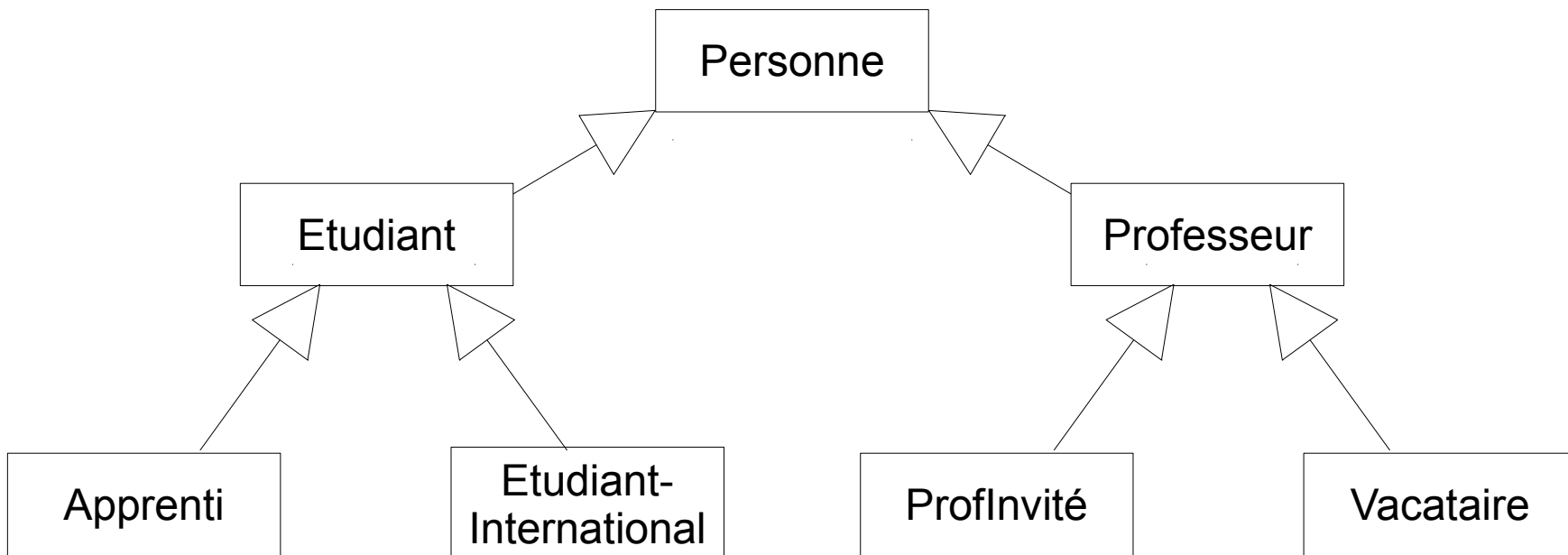
- Relation dont le but est :
 - Réutiliser des classes existantes
 - Simplifier l'écriture de plusieurs classes similaires
 - Simplifier la correction ou la modification de classes
 - Traiter des problèmes de généricité
- Attention : il ne s'agit plus de faire communiquer des objets ou de définir des responsabilités entre objets

Héritage : définition

- Mécanisme de généralisation-spécialisation
 - Spécialisation : modifier une classe pour lui rajouter des fonctionnalités
 - Généralisation : factoriser des caractéristiques communes entre plusieurs classes -> classes plus abstraites

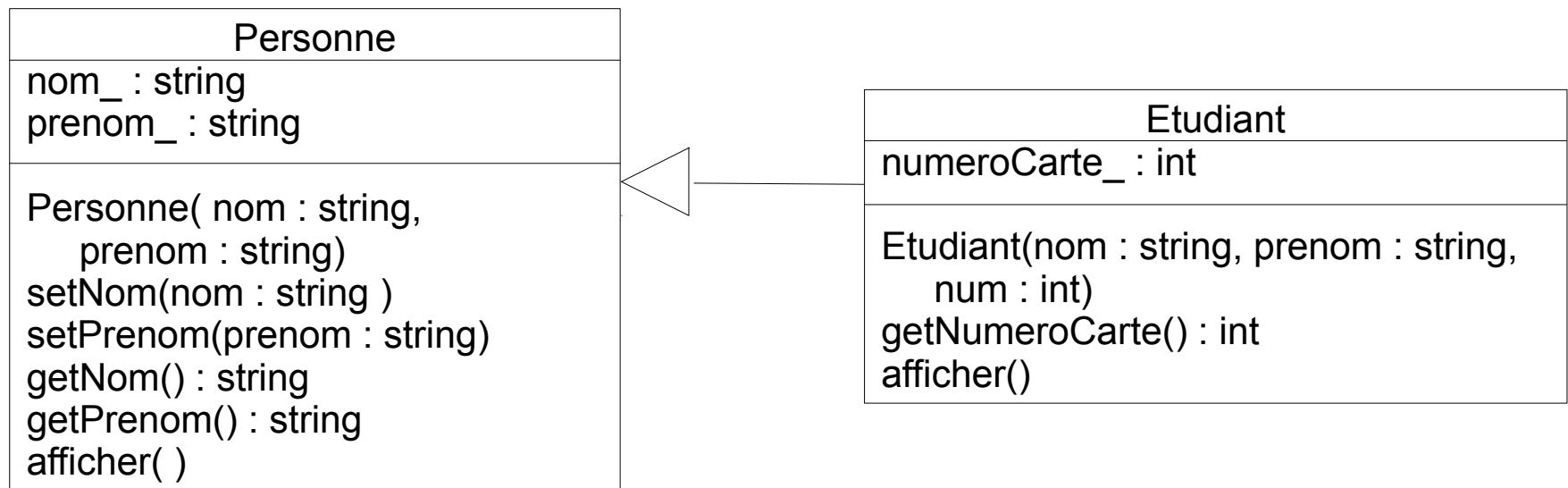
Héritage : définition

- Exprime qu'une classe est :
 - une sorte de ...
 - Etudiant est une sorte de Personne
 - Apprenti est une sorte d'Etudiant



Héritage : mise en oeuvre

- Etudiant est dérivé de la classe Personne :



Déclaration de Personne

```
class Personne
{
    string nom_;
    string prenom_;
public :
    Personne(const string &nom="",const string &prenom="");
    void setNom(const string &nom);
    void setPrenom(const string &prenom);
    const string &getNom() const {return nom_;}
    const string &getPrenom() const {return prenom_;}
    void afficher() const ;
};
```

Déclaration de Etudiant

```
class Etudiant : public Personne
```

Les membres publics
restent publics

Etudiant est dérivé de Personne :
Etudiant possède tous les champs et
méthodes de Personne

```
{  
  int numeroCarte_;
```

```
public :
```

```
  Etudiant(const string &nom, const string &pren, int num);
```

```
  int getNumCarte() const {return numeroCarte_;}
```

```
  void afficher() const;
```

```
};
```

Nouveaux champs et
nouvelles méthodes

Méthode redéfinie

Utilisation de la classe

```
int main()
{
    Etudiant e("Durand","Jean",105432);
    e.afficher();                // Méthode redéfinie prioritaire
    cout<<"Numéro : "<<e.getNumeroCarte()<<endl; // Nouvelle
    e.setNom("Dupont") ;        // Méthode héritée
    e.afficher();                // Méthode redéfinie
    e.Personne::afficher();     // Méthode héritée
    return 0 ;
}
```

Définition du constructeur

```
Etudiant::Etudiant(const string &nom,const string &pren, int num)
    : Personne(nom,prenom)
{
    numeroCarte_=num;
}
```

Le constructeur de Personne sera appelé :

- Avant le constructeur de Etudiant
- Avec les paramètres précisés

Redéfinition de afficher (1/3)

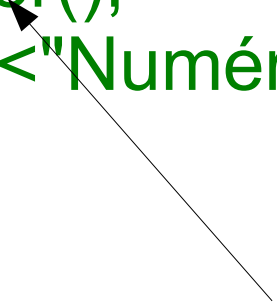
```
void Etudiant::afficher () const
{
    cout<<"Etudiant :"<<endl;
    cout<<"Nom :"<<nom_<<endl;
    cout<<"Prénom :"<<prenom_<<endl;
    cout<<"Numéro de carte :"<<numeroCarte_<<endl;
}
```

Attention : les champs sont privés !

- Il ne sont pas accessibles ici
- Il faut passer par une méthode
- Ou définir les champs protégés dans `Personne`

Redéfinition de afficher (2/3)

```
void Etudiant::afficher () const
{
    cout<<"Etudiant :"<<endl;
    afficher();
    cout<<"Numéro de carte :"<<numeroCarte_<<endl;
}
```



Attention : la nouvelle méthode Afficher est prioritaire !

- Ici on appelle la méthode que l'on est en train de définir : récursion infinie

Redéfinition de afficher (3/3)

```
void Etudiant::afficher () const
{
    cout<<"Etudiant :"<<endl;
    Personne::afficher();
    cout<<"Numéro de carte :"<<numeroCarte_<<endl;
}
```

On donne le nom complet de la méthode

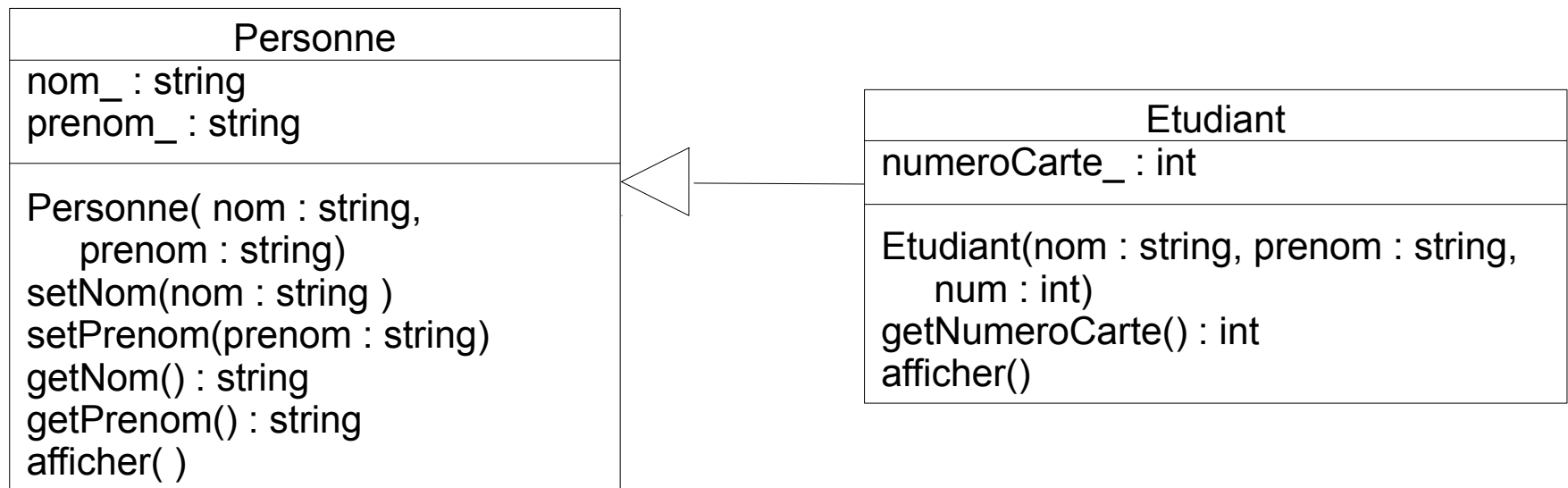


Les méthodes virtuelles

- Mécanisme utilisant l'héritage pour :
 - Manipuler des objets sans connaître leur type
 - Ecrire des méthodes « génériques » qui s'adaptent à de nouvelles classes
- Nous allons voir :
 - Appel de méthode via un pointeur
 - Méthodes génériques
 - Classes abstraites

Appel de méthode via un pointeur

- Rappel des classes Etudiant et Personne



Appel de méthode via un pointeur

- **Exemple**

```
int main()
{
    Personne p("Durand","Jean");
    Etudiant e("Dupont","Frédérique",105433);
    Personne *ptr;
    ptr=&p;
    ptr->afficher();
    ptr=&e;
    ptr->afficher();
    return 0 ;
}
```


Appel de méthode via un pointeur

- Analyse des lignes

```
ptr=&p;           // ptr pointe sur l'objet p (Personne)
```

```
ptr->afficher();  // appel de Personne::afficher() via ptr
```

```
ptr=&e;           // ptr pointe sur l'objet e (Etudiant)  
                // faut-il une conversion de pointeurs ?
```

```
ptr->afficher();  // Quelle méthode afficher ?
```

- Un Etudiant est une Personne !
- L'appel de méthode correspond au type du pointeur !

Appel de méthode via un pointeur

- Comment garder le type réel de l'objet ?
- Utiliser une méthode *afficher* « spéciale »
- On modifie la déclaration de *afficher* dans la classe *Personne* :

```
class Personne
{
...           // Déclaration des champs
public :
...           // Déclaration des méthodes
virtual void afficher() const;
};
```

Appel de méthode via un pointeur

- Définition de la méthode afficher

```
void Personne::afficher() const
{
    cout<<"Personne : "<<Prenom<<" "<<nom_<<endl;
}
```

- Dans la classe Etudiant

- La méthode *afficher* redéfinie est automatiquement virtuelle (si elle a la même signature)
- On peut aussi préciser le mot clé « virtual » mais ce n'est pas obligatoire

Appel de méthode via un pointeur

- Définition de Etudiant::afficher()

```
void Etudiant::afficher () const
{
    cout<<"Etudiant :"<<endl;
    Personne::afficher();
    cout<<"Numéro de carte :"<<numeroCarte_<<endl;
```

Appel de méthode via un pointeur

- Exemple après modification des classes

```
int main()
{
    Personne p("Durand","Jean");
    Etudiant e("Dupont","Frédérique",105433);
    Personne *ptr;
    ptr=&p;
    ptr->afficher();    // Personne::afficher()
    ptr=&e;
    ptr->afficher();    // Etudiant::afficher()
    return 0 ;
}
```

Mécanisme des méthodes virtuelles

- Comment ça marche ?
 - Le compilateur ajoute des informations de type dans la classe
 - Il gère un tableau d'adresse pour chaque méthodes virtuelle
 - L'appel à une méthode virtuelle passe par le tableau d'adresse
- Avantage : on peut manipuler des objets sans connaître leur type

Mécanisme des méthodes virtuelles

- **Avantage** : on peut manipuler des objets sans connaître leur type réel
- **Pas besoin de conversion de pointeurs**
- **Exemple** : tableau de pointeur sur Personne

```
vector<Personne *> tab(4);  
tab[0]= new Personne("Durand","Jean");  
tab[1]= new Etudiant("Dupont","Frédérique",105433);  
tab[2]= new Etudiant("Trucmuche","Alain",105433);  
tab[3]= new Personne("Machin","Alex");  
int i;  
for (i=0;i<4;i++) tab[i]->afficher();    // s'adapte au type
```

Méthodes génériques

- Méthode capable de s'adapter aux classes dérivées sans avoir besoin de la redéfinir
 - Il suffit que cette méthode appelle des méthodes virtuelles qui seront redéfinies
- Exemple : méthode *editerChamps* dans la classe *Personne*
 - Déclarée dans la classe *Personne*
 - Dépend des méthodes *afficher* et *demander*

Définition de la méthode générique

```
void Personne::editerChamps()
{
    char reponse='o';
    while(reponse=='o')
    {
        afficher();
        cout<<"Voulez-vous modifier les champs (o/n)?"
        cin>>reponse;
        if (reponse=='o')
        {
            cout<<"Entrez la valeur de chaque champ" <<endl;
            demander();
        }
    }
}
```

Utilisation pour les classes dérivées

- La méthode s'adapte à la classe

```
int main()
{
    Personne p("Durand","Jean");
    Etudiant e("Dupont","Frédérique",105433);

    p.editerChamps(); // Edition des champs de Personne
    e.editerChamps(); // Edition des champs de Etudiant
    return 0 ;
}
```

Il faut des méthodes virtuelles redéfinies dans les classes dérivées

- Déclaration de la classe Personne

```
class Personne
{
...           // Déclaration des champs
public :
...
virtual void afficher() const;    // Méthode virtuelle
virtual void demander();         // Méthode virtuelle
void editerChamps();             // Méthode générique
};
```

Méthode générique

- Définition des méthodes

```
void Personne::demander()  
{  
    cout<< "Prénom : ";  
    cin>>prenom_;  
    cout<< "Nom : ";  
    cin>>nom_;  
}
```

Méthode générique

- Dans la classe Etudiant :

```
class Etudiant : public Personne
{
    ...                // Déclaration des champs
public :
    ...
    void afficher();    // Automatiquement virtuelle
    void demander();    // Automatiquement virtuelle
};

void Etudiant::demander()
{
    Personne::demander();
    cout<<"Numéro de carte : " ;
    cin>>numeroCarte_;
}
```

Méthode générique

- Comment ça marche ?
- Dans la méthodes EditerChamps, les méthodes virtuelles Afficher et Demander sont appelées via le mécanisme des tables virtuelles

```
void Personne::editerChamps()
{
    ...
    while(reponse=='o')
    {
        afficher();
        ...
        if (reponse=='o') {...
        demander();}
    }
}
```

Classe abstraite

- On pourrait aussi définir *editerChamps* dans une classe servant de base à *Personne*
- Si cette classe ne possède aucun champ, les méthodes *afficher* et *demander* ne seraient pas définies
- La classe serait qualifiée d'abstraite

Classe abstraite

- Déclaration de la classe abstraite Base

```
class Base
{
public :
virtual void afficher() const = 0;      // virtuelle pure
virtual void demander() = 0;           // virtuelle pure
void editerChamps();                   // Méthode générique
};

void Base::editerChamp()
{
...   // Idem classe Personne
}
```


Classe abstraite

- On ne peut pas déclarer d'objets instance d'une classe abstraite
- Les méthodes virtuelles pures *afficher* et *demander*
 - N'ont pas de définition
 - Doivent être obligatoirement redéfinies dans les classes dérivées

Classe abstraite

- Nouvelle déclaration de la classe Personne

```
class Personne : public Base
{
...           // Déclaration des champs
public :
...
    void afficher() const;    // Méthode virtuelle à définir
    void demander();         // Méthode virtuelle à définir
};
```

Exercice 10

Soit la classe Point définie par :

```
class Point
{
    int x_;
    int y_;
public :
    Point(int x=0,int y=0) ;
    void afficher() const;
    int getX() const {return x_;}
    int getY() const {return y_;}
};
```

Exercice 10

```
Point ::Point(int x,int y)
{
    x_=x;
    y_=y;
}
```

```
void Point ::afficher() const
{
    cout<<"Point ("<<x_<<" , "<<y_<<)" ;
    cout<<endl;
}
```

Exercice 10

- a) On veut définir une classe *PointN* dérivée de la classe *Point* qui associe un nom à un point (le nom est un objet string). Elle devra avoir un constructeur et une méthode *setNom* pour changer le nom du point. Donner le diagramme de classe et l'implantation de la classe en C++.
- b) Rajouter une méthode *afficherN* qui affiche les caractéristiques du *PointN* (nom et coordonnées).
- c) Redéfinir la méthode *afficher* pour afficher les caractéristiques d'un *PointN*. Donner un exemple d'utilisation.